



東北大學

KF/EKF/ESKF算法介绍及其在多传感器融合定位中的应用

汇报人: 张洋

学号: 20232411

《数字矿山技术》

2026年04月13日



自强不息 知行合一



自强不息 知行合一

01

KF/EKF/ESKF算法简介

02

关键代码实现示例

03

运行结果和精度分析

04

任务总结与未来展望



東北大學

目录

CONTENTS



示例引入

有一辆小车，从原点出发，以2m/s的速度自西向东做直线运动，t-1时刻在距离原点的东6m处，t时刻雷达测得小车距离原点的东9m处。已知在假定小车做匀速直线运动的前提下，该运动模型的方差为 4 m^2 ，雷达测试仪对距离测试的方差为 9 m^2 ，**小车在距离原点的何处？在距离原点何处的可能性最大？**



2m/s 直线运动

T-1 时刻

T 时刻

原点

6m

? m

我们可以根据运动方程得到小车当前的位置，为 $x_t = 8\text{m}$ ，我们称之为**预测值**。雷达测得的9m，我们称为**观测值**。我们直观的会认为，方差大所对应的值，其权重系数更小，且二者权重系数之和为1，因此，最终值为：

$$\frac{\theta_r^2}{\theta_r^2 + \theta_q^2} \cdot x_t + \frac{\theta_q^2}{\theta_r^2 + \theta_q^2} \cdot z_t = \frac{1}{1+4} \times 8 + \frac{4}{1+4} \times 9 = 8.8(\text{m})$$

事实上，对于一维匀速直线运动模型、观测量只有一个的情况而言，上述就是卡尔曼滤波最优值的**更新公式**，对于二维及以上的模型，有更复杂的表现形式。



公式介绍

卡尔曼滤波是一种最优估计算法。它的核心思想是：把带有噪声的多个信息源（预测和测量）融合起来，得到比任何一个单独信息更准确的估计。

卡尔曼滤波器的操作包括两个阶段：**预测与更新**。在预测阶段，滤波器使用上一状态的，做出对**当前状态的估计**。在更新阶段，滤波器利用对**当前状态的观测值优化在预测阶段获得的预测值**，以获得一个更精确的新估计值。

卡尔曼滤波假设系统是线性的，并且噪声是高斯白噪声。系统用两个方程描述：

状态方程（预测模型）： $\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k$

预测的状态： $\mathbf{x}_k = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$ 状态转移矩阵： $\mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$

预测的新位置 = 旧位置 + 速度 × 时间 + 影响因子 × 控制输入（外部添加的影响） + 过程噪声

观测方程（测量模型）： $\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k$

测量的位置 = 观测矩阵（将状态映射为测量空间） × 状态变量 + 测量噪声



公式介绍

预测过程：状态预测： $\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}_k \mathbf{u}_k$

预测误差： $\mathbf{e}_{k|k-1} = \mathbf{x}_k - \hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \mathbf{e}_{k-1|k-1} + \mathbf{w}_k$

协方差矩阵定义为： $\mathbf{P}_{k|k-1} = E[\mathbf{e}_{k|k-1} \mathbf{e}_{k|k-1}^T]$

协方差预测（先验不确定性）： $\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k$

更新过程：状态更新（后验估计）： $\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1})$

$\mathbf{K}_k$ 是卡尔曼增益，它决定了我们相信预测还是相信测量的程度。选择其值的原则是使后验估计误差的协方差最小（即迹最小、方差和最小）

预测误差： $\mathbf{e}_{k|k} = \mathbf{x}_k - \hat{\mathbf{x}}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{e}_{k|k-1} - \mathbf{K}_k \mathbf{v}_k$

后验协方差： $\mathbf{P}_{k|k} = E[\mathbf{e}_{k|k} \mathbf{e}_{k|k}^T] = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k)^T + \mathbf{K}_k \mathbf{R}_k \mathbf{K}_k^T$

为了求最小 $\mathbf{P}_{k|k}$ ，对 $\mathbf{K}_k$ 求导令其为 0 得： $\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1}$

卡尔曼增益： $\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1}$

协方差更新（后验不确定性）： $\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}$



公式介绍

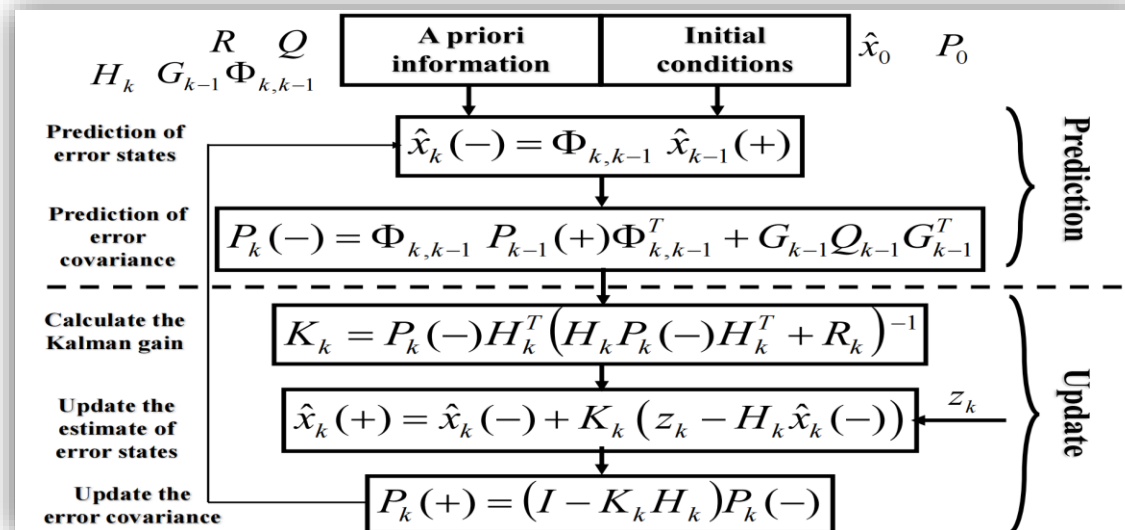
卡尔曼增益: $\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1}$

如果测量噪声 $\mathbf{R}_k$ 很大 (测量不准), 则 $\mathbf{K}_k$ 很小, 更相信预测。

如果先验不确定性 $\mathbf{P}_{k|k-1}$ 很大 (预测不准), 则 $\mathbf{K}_k$ 很大, 更相信测量。

卡尔曼滤波的应用基于以下三个假设前提:

1. 当前时刻状态只和上一时刻状态有关。
2. 模型和系统均满足**线性关系**。
3. 引入的噪声符合高斯分布。



在现实世界中, 绝大多数工程场景都是**非线性系统**。导航中 IMU 与 GNSS 的融合场景, 其状态转移与测量关系都无法用线性方程精准刻画。此时直接套用 KF 会因线性化误差导致估计精度骤降, 甚至出现滤波发散, **扩展卡尔曼滤波 (Extended Kalman Filter, EKF)** 应运而生。



公式介绍

扩展卡尔曼滤波 (EKF) :

$$\text{状态方程: } \mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)$$

$$\text{测量方程: } \mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k)$$

$$\text{状态转移矩阵: } \mathbf{F}_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k}$$

$$\text{观测矩阵: } \mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}}$$

通过对非线性状态方程和观测方程在当前估计值处进行**泰勒展开**, 提取一阶近似的**雅可比矩阵**, 将非线性问题转化为线性问题求解。但这种近似存在明显局限, 当系统非线性程度较高 (如大角度姿态变化) 或长时间运行时, 泰勒展开的误差会急剧累积, 导致估计结果偏离真实值。

误差状态卡尔曼滤波 (ESKF) :

$$\text{状态方程: } \mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k$$

$$\text{测量方程: } \mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k$$

将系统状态拆解为**标称状态与误差状态两部分**: 标称状态遵循原始非线性方程递推, 保留了系统的真实运动特性; 误差状态则是围绕标称状态的微小偏差, 始终满足小量近似条件。ESKF **仅对误差状态进行线性化滤波**, 利用误差状态的卡尔曼滤波预测与更新, 得到微小偏差后再反馈校正标称状态。这种设计不仅规避了大状态线性化的巨大误差。



算法架构



代码实现



结果分析

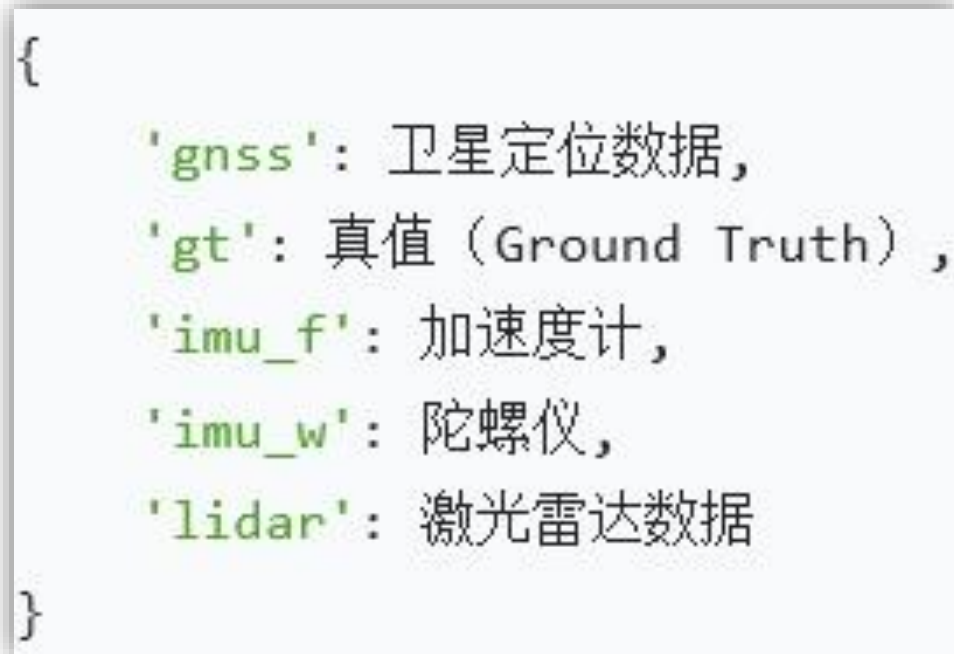
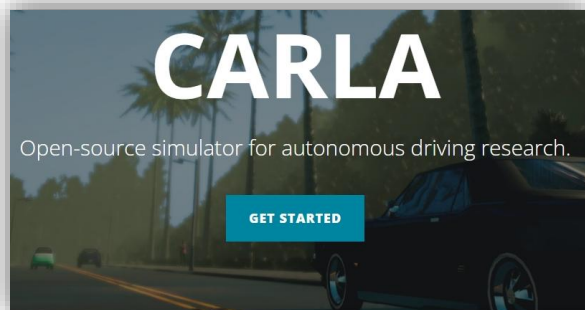


总结展望

代码实现

本次程序使用自动驾驶仿真平台 **CARLA** 生成的标准测试数据集，以 .pkl 格式存储，包含**真实轨迹**（Ground Truth）、**IMU 加速度与角速度**、**GNSS 定位数据**、**LiDAR 点云定位数据**。

代码实现了 **IMU/GNSS/LiDAR 多传感器融合定位**，实现并对比标准卡尔曼滤波（KF）、扩展卡尔曼滤波（EKF）、误差状态扩展卡尔曼滤波（ESKF/ESEKF）三种算法的定位性能。





算法架构



代码实现

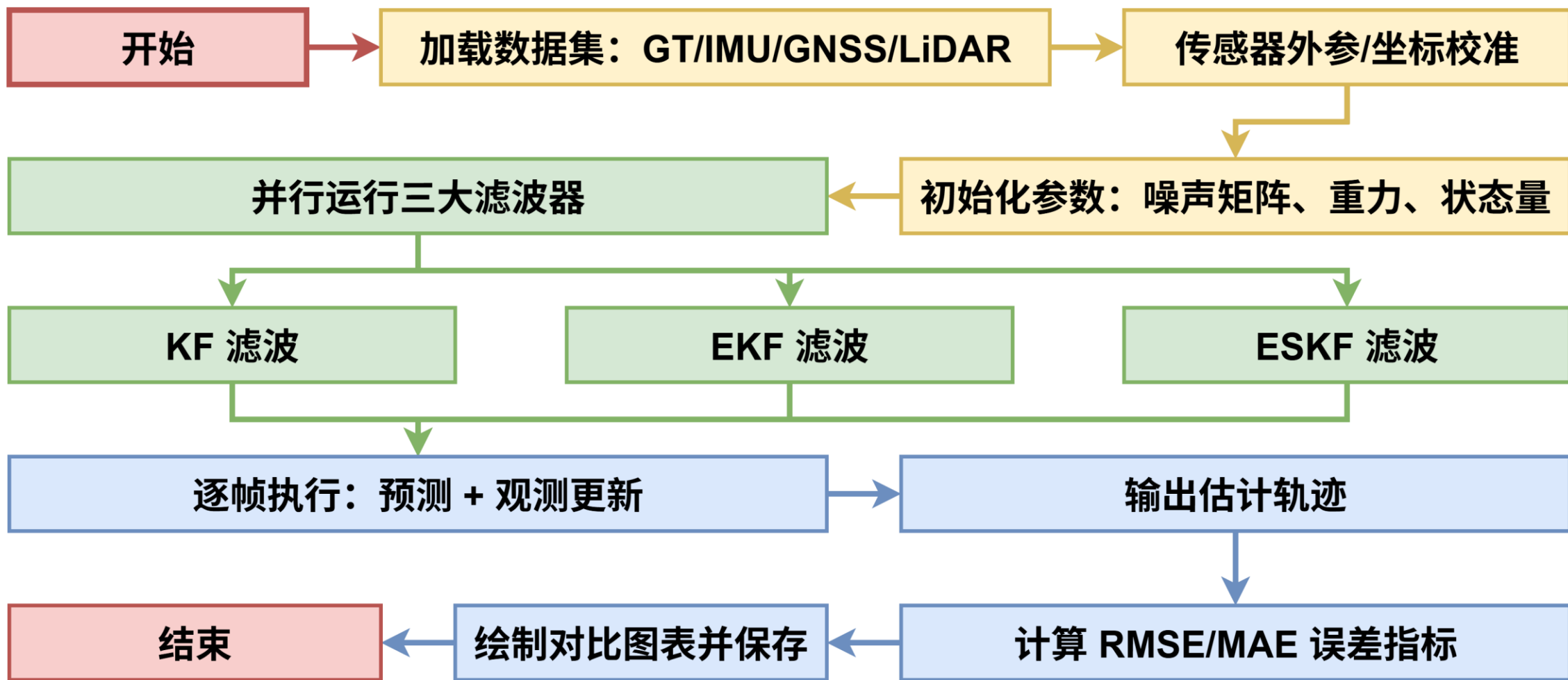


结果分析



总结展望

代码实现





代码实现

```
文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R) ... ESEKF_IMU_GNSS_Lidar-master
es_ekf.py compare_filters_v2.py position_errors_xyz.py compare_filters_v2.py compare_filters_v2.py

127 def run_kf():
128     """标准卡尔曼滤波器 - 假设系统线性"""
129     p_est = np.zeros([imu_f.data.shape[0], 3])
130     v_est = np.zeros([imu_f.data.shape[0], 3])
131     p_cov = np.zeros([imu_f.data.shape[0], 9, 9])
132     p_est[0] = gt.p[0]
133     v_est[0] = gt.v[0]
134     p_cov[0] = np.eye(9) * 0.01
135     gnss_i = 0
136     lidar_i = 0
137
138     def measurement_update_kf(sensor_var, p_cov_check, y_k, p_check, v_check):
139         R = sensor_var * np.eye(3)
140         S = h_jac @ p_cov_check @ h_jac.T + R
141         K = p_cov_check @ h_jac.T @ np.linalg.inv(S)
142         delta_x = K @ (y_k - p_check)
143         p_hat = (p_check + delta_x[:3]).flatten()
144         # v_check已经是1D, 创建列向量进
145
188 def run_ekf():
189     """扩展卡尔曼滤波器 - 处理非线性旋转"""
190     p_est = np.zeros([imu_f.data.shape[0], 3])
191     v_est = np.zeros([imu_f.data.shape[0], 3])
192     q_est = np.zeros([imu_f.data.shape[0], 4])
193     p_cov = np.zeros([imu_f.data.shape[0], 9, 9])
194     p_est[0] = gt.p[0]
195     v_est[0] = gt.v[0]
196     q_est[0] = Quaternion(euler=gt.r[0]).to_numpy()
197     p_cov[0] = np.eye(9) * 0.01
198     gnss_i = 0
199     lidar_i = 0
200
201     def measurement_update_ekf(sensor_var, p_cov_check, y_k, p_check, v_check, q_check):
202         R = sensor_var * np.eye(3)
203         S = h_jac @ p_cov_check @ h_jac.T + R
204         K = p_cov_check @ h_jac.T @ np.linalg.inv(S)
205         delta_x = K @ (y_k - p_check)
206         p_hat = (p_check + delta_x[:3]).flatten()
207         delta_phi = delta_x[3:6]

63 def run_eskf():
64     """运行ESKF算法"""
65     p_est = np.zeros([imu_f.data.shape[0], 3])
66     v_est = np.zeros([imu_f.data.shape[0], 3])
67     q_est = np.zeros([imu_f.data.shape[0], 4])
68     p_cov = np.zeros([imu_f.data.shape[0], 9, 9])
69     p_est[0] = gt.p[0]
70     v_est[0] = gt.v[0]
71     q_est[0] = Quaternion(euler=gt.r[0]).to_numpy()
72     p_cov[0] = np.zeros(9)
73     gnss_i = 0
74     lidar_i = 0
75
76     def measurement_update(sensor_var, p_cov_check, y_k, p_check, v_check, q_check):
77         R = sensor_var * np.eye(3)
78         K_k = p_cov_check @ h_jac.T @ np.linalg.inv(h_jac @ p_cov_check @ h_jac.T + R)
79         delta_xk = K_k @ (y_k - p_check)
80         p_hat = (p_check + delta_xk[:3]).flatten()
81         delta_phi = delta_xk[3:6]
```



算法架构



代码实现



结果分析



总结展望

代码实现

运行环境 **Python 3.11** 版本, 需要安装以下库依赖:

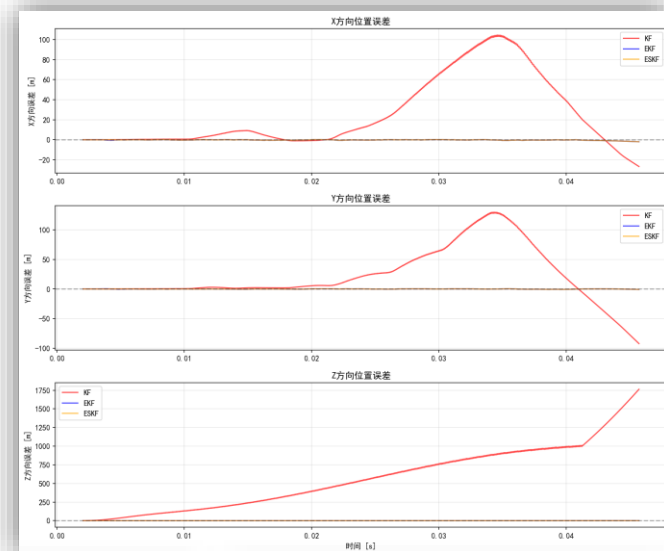
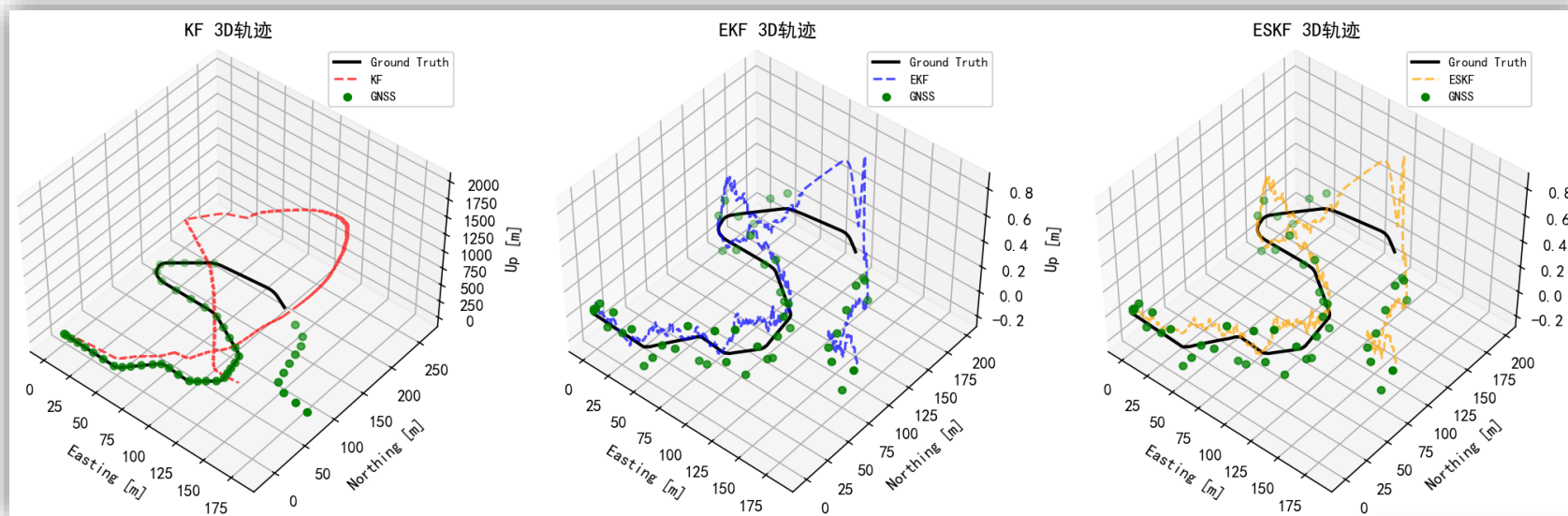
库名	用途
numpy	核心数值计算 (矩阵运算、数组操作、线性代数求解等)
matplotlib	可视化绘图 (3D 轨迹、误差曲线、条形对比图等)
pickle	内置库, 无需安装, 用于加载.pkl 格式的数据集
pathlib	内置库, 无需安装, 用于路径处理 (跨平台文件路径管理)

输出结果主要为可视化数据, 主要包含以下**五张图表**:

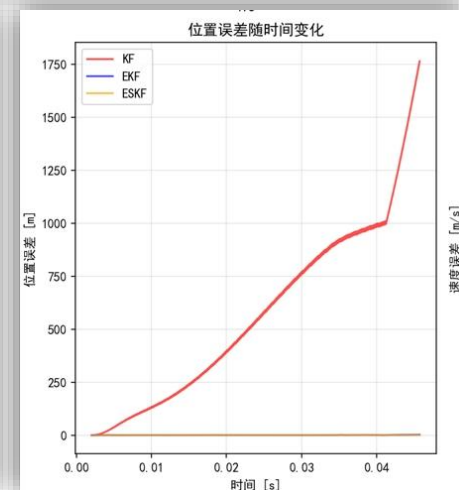
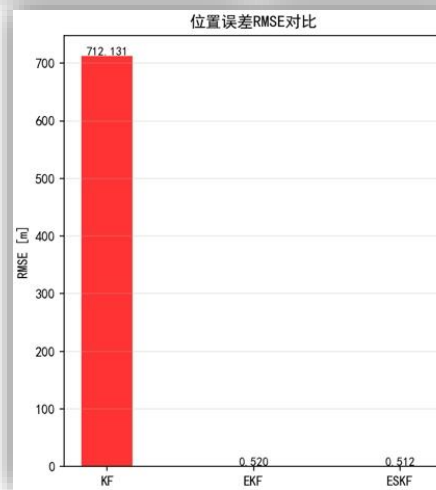
图表类型	用途
3D 轨迹对比 (KF/EKF/ESKF)	直观展示估计轨迹与 GT 真值、GNSS 测量点的偏差
位置 / 速度误差时序曲线	分析误差随时间的变化趋势, 对比各滤波器的稳定性
X/Y/Z 方向误差分解	定位误差主要来源 (东向 / 北向 / 竖向)
EKF vs ESKF 专项对比	聚焦两种非线性滤波器的性能差异, 量化改进幅度
误差指标条形图	定量对比 RMSE/MAE, 添加数值标签, 便于论文 / 报告引用



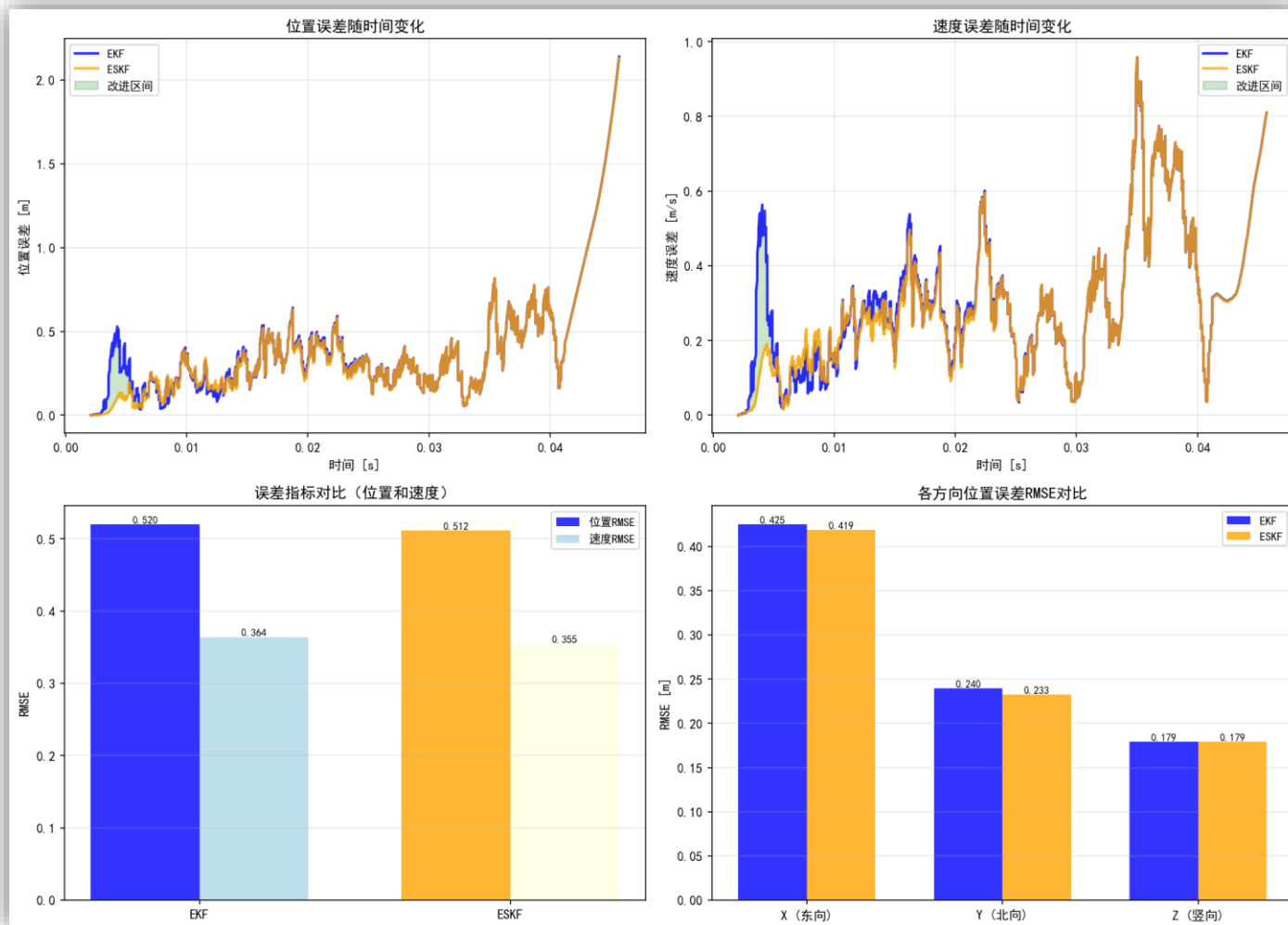
结果分析



3D图中，KF 因忽略姿态非线性与重力项，轨迹与真值偏离最明显；EKF 和 ESKF 轨迹与黑色真值线高度贴合。时序误差图和柱状图显示 KF 误差持续发散，EKF 和 ESKF 误差始终保持最低且平稳。KF 在三轴均呈现大幅误差波动与漂移，Z 轴受重力项缺失影响误差尤为突出；EKF 和 ESKF 三轴误差有所收敛但仍存在明显波动。



结果分析

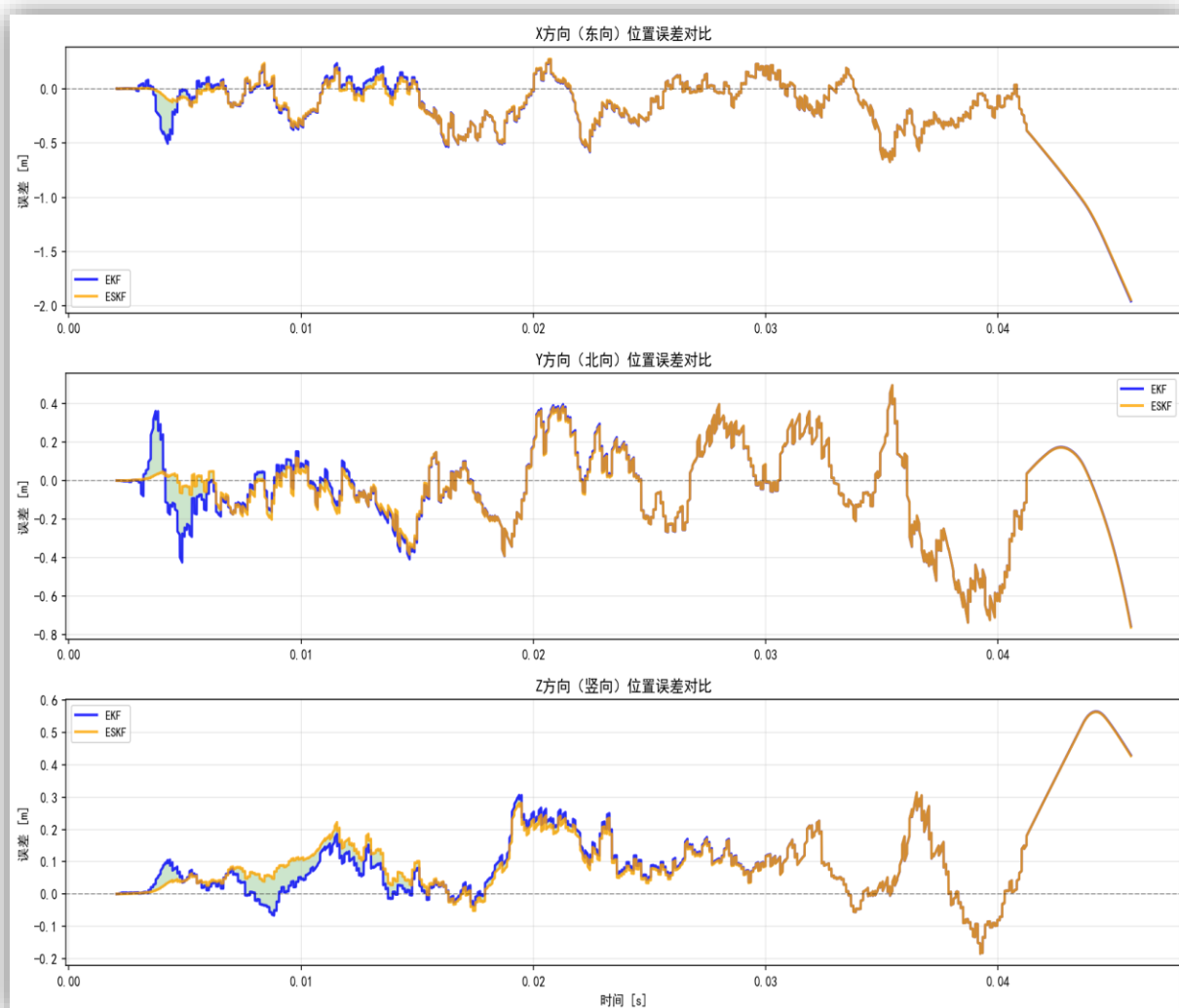


该图为 EKF 与 ESKF 的精细化误差对比，包含位置 / 速度时序误差、整体 RMSE、XYZ 三轴 RMSE。

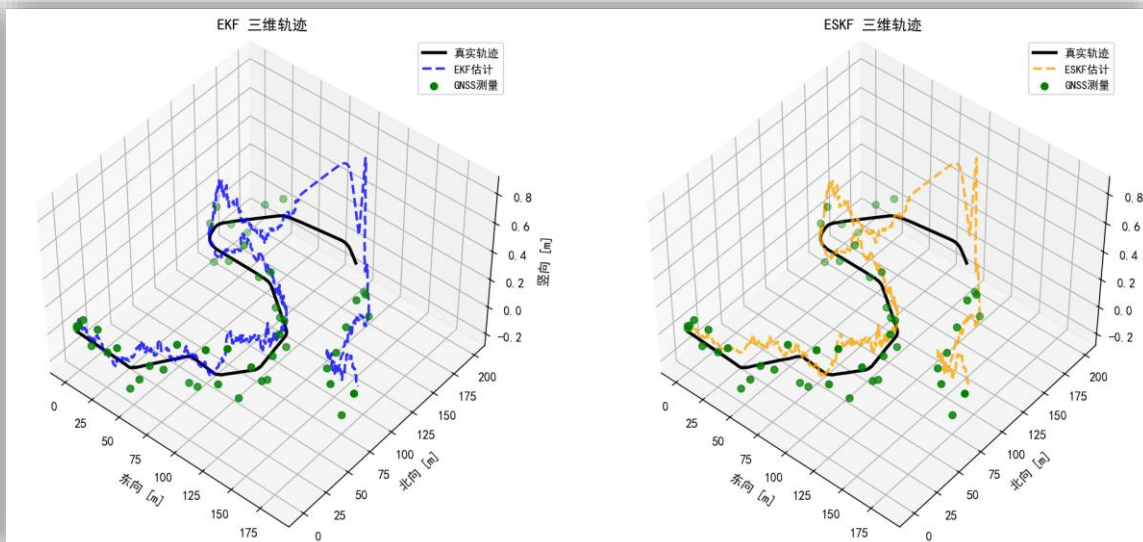
时序子图中 ESKF 误差均低于 EKF，在EKF波动较大的区域，ESKF均表现出较好的稳定性。

柱状子图定量显示 ESKF 位置与速度 RMSE 均优于 EKF，XYZ 三轴均实现均衡精度提升，综合性能领先。

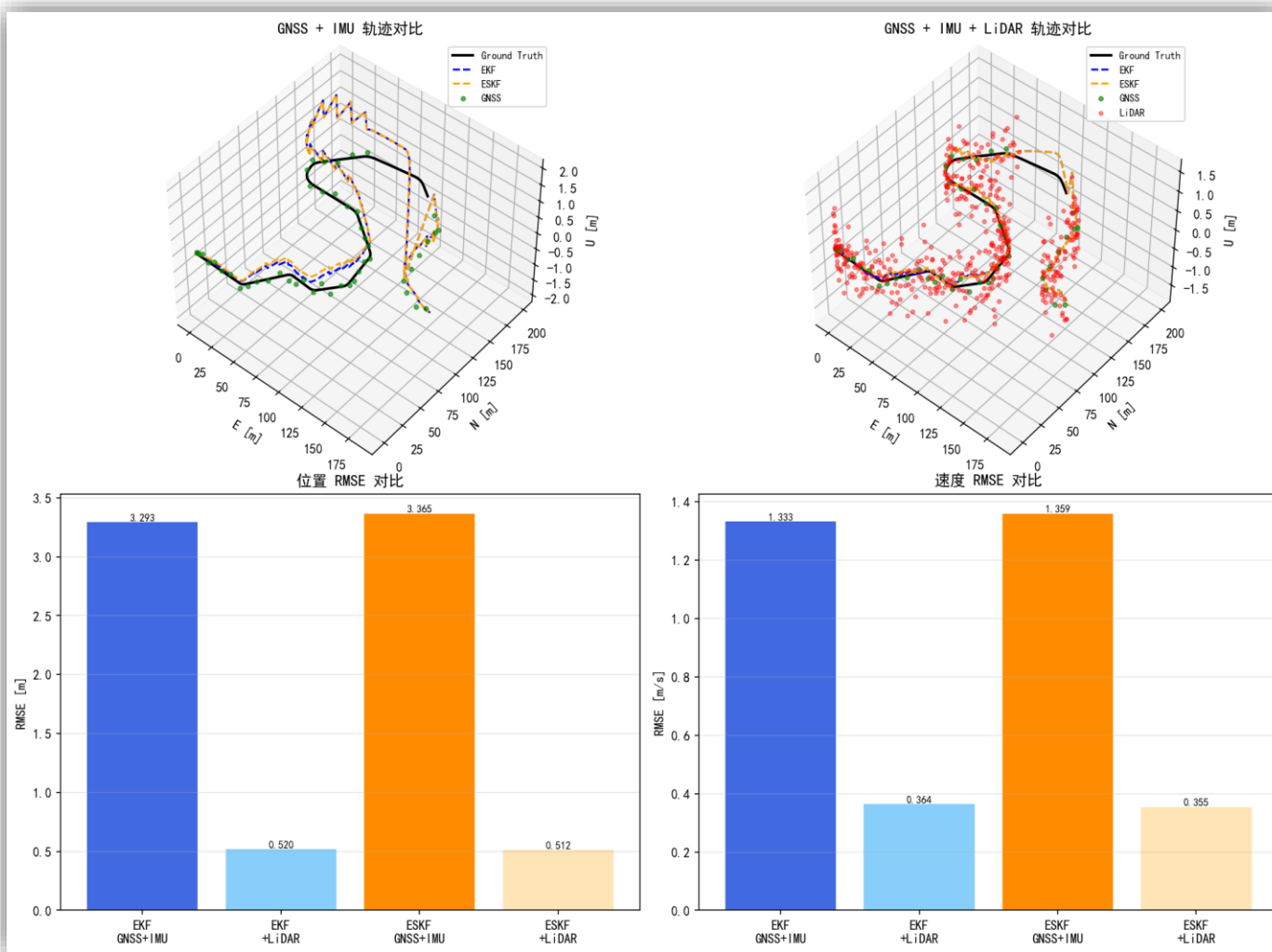
结果分析



左侧 EKF 与右侧 ESKF 3D 轨迹区别不大。但 ESKF 在三轴均能**快速收敛并维持低误差**，EKF 则存在持续波动与漂移，尤其在动态变化段差异更显著，充分验证 ESKF 在多维姿态与位置估计上的优越性。



结果分析



在仅使用 GNSS+IMU 的场景中，EKF 与 ESKF 的轨迹均能大致贴合真实轨迹，但存在明显的漂移与偏差，其中 ESKF 的轨迹发散程度略高于 EKF；而引入 LiDAR 观测后，两种算法的轨迹精度均得到了质的提升，EKF+LiDAR 与 ESKF+LiDAR 的轨迹几乎与真实轨迹完全重合，LiDAR 的稠密点云观测有效修正了 GNSS/IMU 的累计误差，显著提升了定位鲁棒性。

综合来看，LiDAR 的引入是定位精度提升的核心关键，其观测有效弥补了 GNSS/IMU 组合的漂移问题。



算法架构



代码实现



结果分析



总结展望

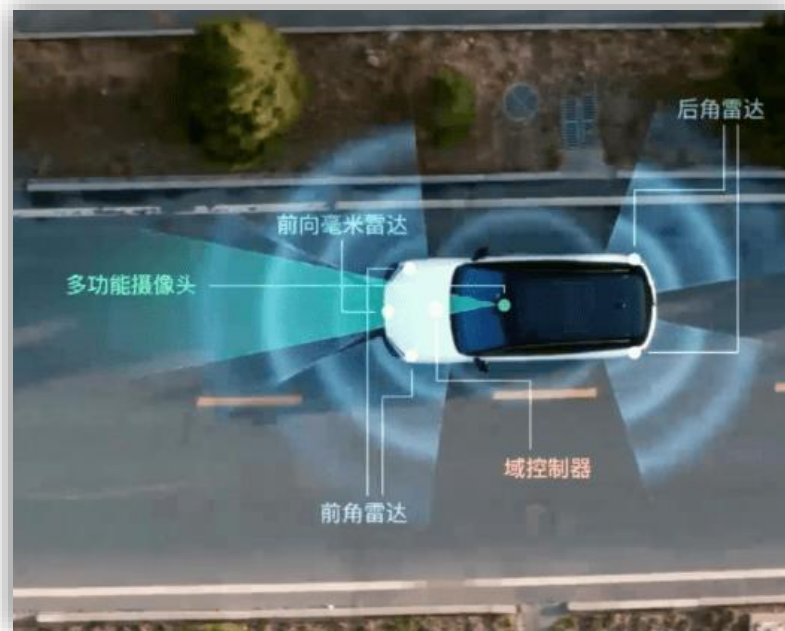
对比维度	卡尔曼滤波 (KF)	扩展卡尔曼滤波 (EKF)	误差状态卡尔曼滤波(ESKF)
核心模型	线性高斯系统	非线性系统（全状态建模）	非线性系统 (标称状态 + 误差状态拆分)
线性化方式	无	对全状态做一阶泰勒展开	仅对小量误差状态做线性化
定位精度	极低	中等	高
数值稳定性	稳定	较差	优秀
计算复杂度	最低	中等（需计算雅可比矩阵）	中等（与 EKF 相当）
适用场景	线性、高斯、 无强非线性的简单系统	弱非线性、状态变化 平缓的导航场景	强非线性、高动态的自动 驾驶 / 机器人 / 测绘定位
核心优势	原理简单、计算高效	实现门槛低， 经典非线性滤波方案	精度高、鲁棒性强， 工程化落地性好
核心局限性	无法适配导航非线性场景	强非线性下线性化误差大， 易发散	状态拆分逻辑复杂， 实现难度略高



总结展望

本次工作完成了 KF、EKF、ESKF 三种滤波算法在 IMU / GNSS / LiDAR 多传感器融合定位中的全流程实现与对比验证。

实验表明：标准 KF 因线性模型局限，在非线性定位场景中误差严重发散；EKF 通过一阶线性化适配非线性系统，定位精度大幅提升；ESKF 采用误差状态分离结构，彻底抑制线性化误差，定位精度与鲁棒性最优。LiDAR 的引入是定位精度提升的核心关键。



未来，多传感器融合定位将成为无人驾驶的核心支撑技术。通过 IMU、GNSS、LiDAR、视觉等多源信息深度融合，可实现**全天候、全场景、高鲁棒性**的精准定位，为无人驾驶的路径规划、决策控制、安全行驶提供底层定位保障。



東北大學

汇报完毕，请老师批评指正！

THAT'S ALL FOR MY PRESENTATION. THANK YOU!

汇报人: 张洋

学号: 20232411

《数字矿山技术》

2026年04月13日



自强不息 知行合一